

Memory as a Learned Policy

A research proposal for compounding-knowledge AI agents

Parth Kocheta

University of Maryland | pranayko021@gmail.com

Submitted to the Pace Fellowship, Summer 2026

The thesis

The most important constraint on AI-for-science is not model intelligence. It is memory. A research scientist running a six-month investigation accumulates context across hundreds of papers, dozens of failed experiments, and a steadily evolving model of which lab techniques work in their hands. A frontier language model running the same investigation forgets what it read on Monday by Wednesday. The compounding asset of a scientific career—knowing more about your specific subfield each week than you did the week before—has no operating system in current AI agents.

Every shipping memory system today (Mem0, Letta, Mastra, Zep, my own prior knowledge-graph system) pairs an external store with a hand-tuned control layer that decides what to write and how to read. The store grows from use; the control layer does not. I think this is the layer that needs to be *learned*.

The proposal

I am training two LoRA adapters on a 4B-parameter open-source language model that share an op vocabulary across read and write sides of memory. The *write policy* ingests one conversation turn at a time and emits operations into a typed knowledge graph—creates, updates, marks of contradiction, merges, archivals. The *read policy* retrieves from the same graph and answers questions. Both policies are trained with GRPO [1] (DeepSeek’s critic-free RL algorithm) on the LoCoMo [2] and LongMemEval [3] benchmarks of long-horizon conversational memory.

The hard problem is the write reward. Outcome supervision is sparse and delayed: a write op only matters if the read policy can later use what was stored. My contribution is a dense companion signal I call *evidence coverage*—the fraction of dialogue turns required to answer a held-out question that the policy actually preserved in the graph, computed deterministically from existing benchmark annotations and the graph’s provenance fields. Coverage carries 60% of the reward; an LLM-judge-anchored answer-correctness term carries 40%. The mix trains $\sim 6\times$ faster per step and is robust to judge noise.

The same trained adapter compiles to three storage backends: a flat chunk store, a typed-transition knowledge graph, and a Mastra-style observation log. Backend transfer (one policy, three stores) is the empirical claim that lets this work be a *layer* rather than a *system*.

Why this matters at the intersection Pace cares about

Infrastructure. Long-horizon agents are bottlenecked on context window economics. A learned memory operating system pushes the expensive recall work into a small adapter (~ 200 MB) and a substrate-agnostic vocabulary, decoupling the agent’s intelligence from the linear-in-history cost of every retrieval. This is the missing layer between today’s RAG stacks and a real “personal compute substrate.”

Economics. The value of an AI collaborator scales with how much of your specific context it has compounded. Today the practical ceiling is the context window. Cracking the memory-OS problem changes the unit economics of customer support, sales engineering, scientific research, and personal AI: the marginal hour with a user gets cheaper because the model accumulates instead of re-deriving. Whoever owns the adapter gets a network effect that the foundation-model layer does not.

The physical world. The benchmark-shaped reward I rely on (LoCoMo’s per-question evidence labels) is a research crutch. Real deployments have no such labels. The fellowship period would let me investigate two routes to closing that gap: (a) weak evidence mining where an LLM extracts dia-id labels from a session transcript at end-of-conversation, and (b) next-turn implicit reward where the user’s follow-up message scores the prior turn’s stored facts. Either route turns the technique into something that can run in a deployed system where users do not annotate.

Status

The full system is implemented and committed to a public repository. The write/read environments, the dense reward, the three backends, and the co-training outer loop are all production code. Smoke runs through Tinker (Thinking Machines’ managed RL training service) confirm end-to-end training, with coverage-mean of 0.77 and judge accuracy of 0.55 on a tiny 5-step smoke. The full Phase B run is queued. A 15-page draft of the long-form paper exists; this proposal is the compressed version.

What I would do at Pace

The fellowship period would convert two pieces of this work into publishable artifacts: (1) the empirical paper with the full backend- transfer table on LoCoMo and LongMemEval, and (2) a short essay arguing that memory is the next vertical stack of LLM-agent infrastructure worth investing in—with a clear taxonomy of what is durable (substrate-agnostic adapters), what is interchangeable (the storage itself), and what gets disrupted (per-customer fine-tuning).

All code, paper, and run logs are public: github.com/pkempire/pie.

References

- [1] Z. Shao et al. “DeepSeekMath: Pushing the limits of mathematical reasoning in open language models.” *arXiv:2402.03300*, 2024.
- [2] A. Maharana et al. “Evaluating very long-term conversational memory of LLM agents.” *arXiv:2402.17753*, 2024.

- [3] D. Wu et al. “LongMemEval: Benchmarking chat assistants on long-term interactive memory.” *arXiv:2410.10813*, 2024.